

## B.3 Lab 3: Root Finding Methods

### Problem 1: Bacterial Chemotaxis

Recall that our previous model for a bacteria population assumes that the population's rate of change with respect to time,  $y'(t)$ , is proportional to the current population  $y(t)$ , which led us to the following differential equation model:

$$y'(t) = k * y(t)$$

where  $k$  is the growth constant. However, this model assumes that the population is isolated from the outside world. What if additional bacteria cells migrate to our observation area in search of more favorable conditions? This process is referred to as bacterial chemotaxis.<sup>6</sup>

If we assume the immigration rate,  $b$ , is constant then our model becomes

$$y'(t) = k * y(t) + b.$$

The closed form solution for this differential equation defines our new population function:

$$y(t) = c * e^{kt} + \frac{b}{k} (e^{kt} - 1)$$

where  $c$  is the initial population size at time zero. For this exercise, we are going to use root finding to recover a bacteria's growth constant,  $k$ , from measured data.

In order to simplify things, let's assume that the initial population size and immigration rate are known. Let's further assume that our measured data will always be acquired at time  $t = 1$  hour. Under these assumptions, we can reformulate our population model as a function of the unknown growth constant:

$$y(k) = c * e^k + \frac{b}{k} (e^k - 1).$$

**Problem 1.1:** Write a function called `bacteriaPop` that takes three inputs: 1) a growth constant  $k$ ; 2) the initial population size  $c$ ; and 3) the immigration rate (per hour)  $b$ . Your function should have two outputs: 1) the population size after one hour  $y(k)$ ; and 2) the derivative of the population function,  $y'(k)$ , evaluated at the input  $k$  (Hint: You are going to need the quotient rule). You will include `bacteriaPop` as a subfunction in Problem 1.2's `growthRateRecover.m`.

**Problem 1.2:** Write a function named `growthRateRecover.m` that takes three inputs: 1) the initial population size  $c$ ; 2) the immigration rate (per hour)  $b$ ; and 3)  $N$  the size of the population after one hour. The function should use Newton's method to recover the growth constant using a  $10^{-6}$  tolerance applied to the approximate relative error. Additionally, the function should use `semilogy` to plot the approximate relative error as a function of iteration. The title of this plot should contain the recovered growth rate value. Use 1.0 as the initial guess for Newton's method. Take a look at `growthRateRecover_outline.m`. It sketches out how one might approach problems 1.1 and 1.2.

**Problem 1.3:** We have a *Streptococcus A* infection containing 1024 cells. One hour later, we observe that 102 cells migrated to the infection and a total of 2212 cells are present. Use `growthRateRecover.m` to determine the growth rate.

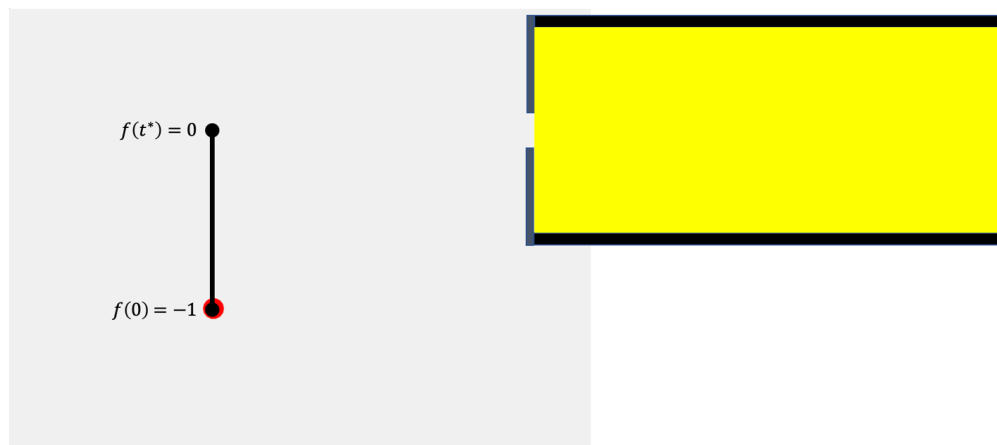
### Problem 2: Beam-On Time

As we discussed in lecture, respiratory gating is one approach for reducing radiation dose to healthy lung during cancer radiotherapy for lung cancer.<sup>7</sup> The idea is to only turn the beam on when the

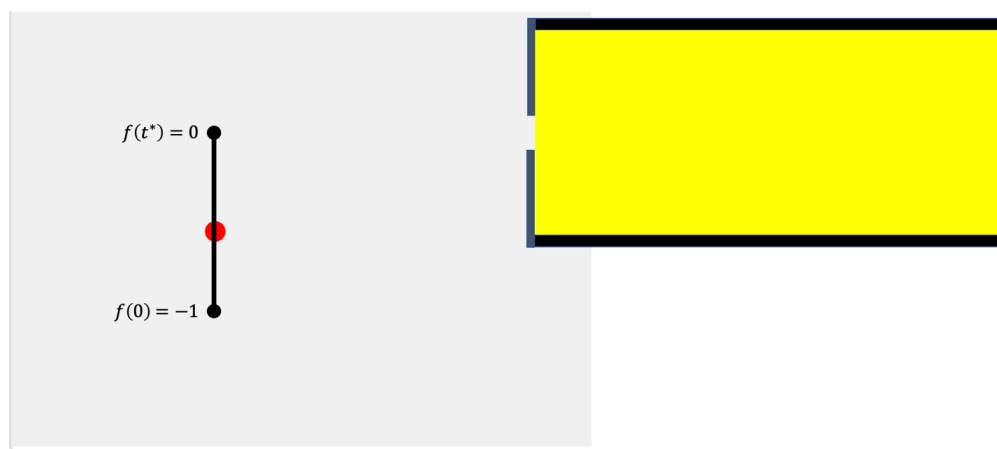
<sup>6</sup><https://www.nature.com/articles/nrm1524>

<sup>7</sup><https://moffitt.org/treatments/radiation-therapy/respiratory-gating/>

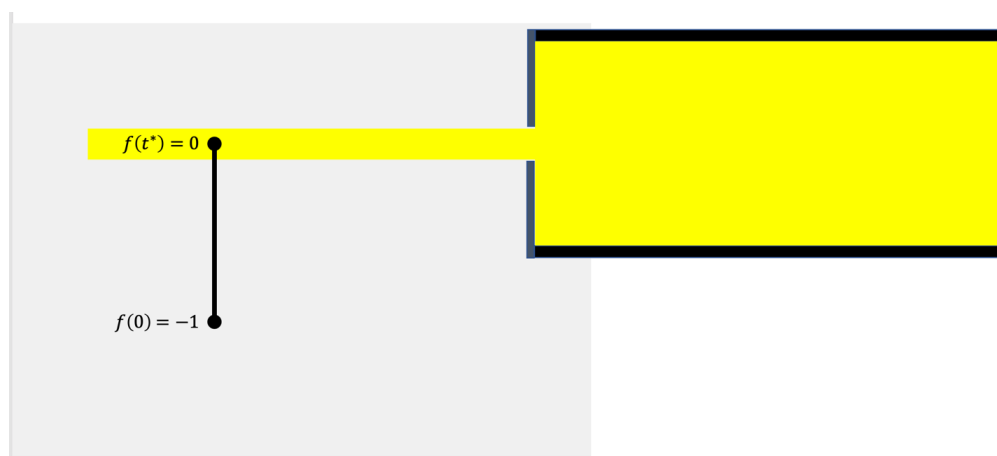
tumor's motion is minimal, such as when the lung is at exhale. Let's use the following simplified model:



The position of the target (tumor) on the black line at time  $t$  is given by the function  $f(t)$ .



We want to turn the beam on when the target is at a height of zero.



At what time,  $t^*$ , do we turn on the beam? To find out, we need to solve a root finding problem:

$$f(t^*) = 0.$$

Though we do not have a closed form representation for the tumor trajectory  $f(t)$ , we do have access to `positionEval.p` which takes as input a query time,  $t$ , and returns the position  $f(t)$ .

`positionEval.p` requires a computationally intensive algorithm to evaluate  $f(t)$ . As such, we need to be mindful of the number of iterations required when settling on an approach to solve this root-finding problem. This issue is common in real-world applications where one desires the roots of a “black-box” function that is computationally expensive to evaluate.

**Problem 2:** Write a function `beamOnTime.m` that calculates the beam-on time,  $t^*$ , such that  $f(t^*) = 0$ . You are free to solve this problem using whatever root finding method you think is best. Your function will need to call `positionEval.p` to evaluate  $f(t)$ . For example:

```
>> val = positionEval(0.5)

val =

    -0.5206
```

provides the position of the tumor at  $t = 0.5$ . Your solution should satisfy a  $10^{-6}$  tolerance applied to the approximate relative error. In addition to the normally required information, the header for your function should describe, in detail, the approach used to calculate  $t^*$ . Regardless of method, the function should use `semilogy` to plot the approximate relative error as a function of iteration. The title of this plot should contain the recovered beam-on time.

## General Submission Guidelines

|                                                          |                                              |
|----------------------------------------------------------|----------------------------------------------|
| <code>[FirstName][LastName]_[filename].m</code>          | For programs                                 |
| <code>[FirstName][LastName]_[filename]_plot_1.pdf</code> | For corresponding plots (export them to pdf) |
| <code>[FirstName][LastName]_[filename]_plot_2.pdf</code> | For problem with more than one plot          |

## Submission Checklist

- Problem 1.1: `bacteriaPop` function
  - Fully commented code
  - Correct population function implementation
  - Correct derivative of population function implementation
- Problem 1.2: `growthRateRecover.m`
  - Fully commented code
  - Correct implementation of Newton’s method
  - Correct stopping criteria
- Problem 1.3
  - Semilogy plot
  - Correct solution
- Problem 2: `beamOnTime.m`
  - Fully commented code complete with method description
  - Semilogy plot showing approximate relative error and iterations
  - Correct solution